

### Columnar-based databases

- Unlike row-based databases, where data is stored as rows on the disk, columnar databases store data as columns, thereby optimising for queries against columns. Hence, they find good use for analytical queries. These databases are often used as data warehouses for business intelligence and reporting. Examples are Apache Cassandra and AWS Redshift.
- These databases are not optimal for transactional workloads that demand high concurrency as they are generally optimised for read-heavy workloads. They could also be an overkill when your data set is small and doesn't demand an analytical set of use cases but rather needs ACID properties. The read cost increases based on how many columns you like to pick in the query.

### Document-oriented databases

- These databases store data in self-describing documents in JSON/BSON or any custom form that is flexible/dynamic in schema. The structure of documents can be easily extended over time, and each key can have its own structure of the document. General use case is storing various attributes against a domain entity. They are indexed using one or more keys or even using attributes for a few technologies.
- Examples are MongoDB and CouchDB; almost every modern-day application can be found using these databases.
- You can choose between technologies based on consistency and availability needs, as discussed with respect to the CAP theorem earlier in this article. Examples include MongoDB and AWS DynamoDB.

### Key-value databases

- These are simple databases used to store values against a key. The key spaces are highly partitioned and horizontally scalable, making these best suited for high throughput, low latency reads and write use cases.
- Examples of key-value databases include Redis, Riak DB, and AWS DynamoDB. A few databases serve a dual purpose – both as a key-value store and as a document database. For instance, AWS DynamoDB can be used as a document database (to store a document against a key) or even a simple value against a key.

### Graph databases

- These databases model data in the form of a graph, and use nodes and edges. These nodes and edges can have properties. They are well suited for modelling highly connected data and conducting complex multi-hop traversals, reverse lookups and relationship analysis which is either too complex with other databases or even impossible to achieve.
- Examples include Neo4j, AWS Neptune, and Azure Cosmos DB.

### Time series databases

- These are designed for efficient storage, retrieval and analysis of timestamp-based information like sensor data, system metrics like CPU, memory usage, financial market data, etc. Trying to

achieve these use cases with other databases will mean conducting complex queries or incurring a lot of I/O cost, which is where time-series databases come to the rescue.

- Examples include Influx DB and Open TSDB.

### In-memory databases

- For high performance/low latency applications that demand microsecond range of read/write latency (as opposed to other databases that usually deliver milliseconds range performance), in-memory databases are the choice. The improved performance is because data is stored and accessed from the primary memory/RAM as opposed to another database that involves disk I/O.
- The downside over other databases is the cost. Because primary memory is generally more costly than disk, one pays a higher cost for the benefit of low latency while choosing in-memory databases. Examples include SAP HANA and Redis.

Besides these, there are several other database types such as ledger databases, spatial databases, and vector databases that are popular for large language models or LLMs and have made a mark in different fields. I've covered just the top commonly used database types for sake of the article's length.  END

### References

- <https://aws.amazon.com/startups/start-building/how-to-choose-a-database/>
- <https://www.geeksforgeeks.org/how-to-choose-the-right-database-for-your-application/>
- 'System Design Interview, An Insider Guide' by Alex Hu.

### By: Shravan I.V.

The author is currently working as a software developer and architect. He is interested in open source technologies, and loves reading about them and experimenting with them during his free time.